HOUSE PRICE PREDICTION PROJECT

Introduction

House Price Prediction is a Machine Learning project used to estimate the price of houses based on various features. It is a regression problem where the output is a continuous value. This project helps in understanding how factors like size, condition, and year built affect house prices. Machine Learning models analyze historical data to make predictions. It is widely used in the real estate industry. This project demonstrates data-driven decision-making. Multiple algorithms are applied to compare performance. The main goal is to build an accurate prediction model. It also improves knowledge of preprocessing and evaluation techniques.

Dataset Description

The dataset used is kc_house_data.csv, which contains housing-related information. Each row represents a house. The dataset includes features like bedrooms, bathrooms, square footage, and year built. It contains numerical data suitable for regression analysis. It is widely used for Machine Learning practice. Some preprocessing is required before training models. The dataset is large enough for testing multiple algorithms. It helps understand real-world data patterns.

Input and Output Variables

Input Features (X)

Input Features Description

1. **Bedrooms**
   Number of bedrooms available in the house. More bedrooms generally increase the house value.
2. **Bathrooms**
   Total number of bathrooms in the house. Houses with more bathrooms are usually priced higher.
3. **Sqft Living**
   The total living area of the house is square feet. This is one of the most important factors affecting price.
4. **Sqft Lot**
   Total land area (lot size) in square feet. Larger plots tend to have higher value.
5. **Floors**
   Number of floors in the house. Multi-floor houses may have higher market value.
6. **Condition**
   Overall condition of the house (usually rated on a scale). Better conditions lead to higher prices.
7. **Grade**
   Represents construction quality and design. Higher grade indicates better quality and higher price.
8. **Sqft Basement**
   Basement area in square feet. Additional usable space increases property value.
9. **Year Built (yr_built)**
   The year the house was constructed. Newer houses are generally more expensive.
10. **Year Renovated (yr_renovated)**
    The year when the house was last renovated. Recently renovated houses often have higher prices.

Output Variable

- **Price**
  The predicted selling price of the house based on the above

Data Preprocessing
Data preprocessing is an important step in Machine Learning. The dataset is loaded using Pandas. Missing values are handled if present. Features and target variables are separated. Data is split into training and testing sets. This ensures unbiased evaluation. Preprocessing improves accuracy. It removes inconsistencies in data. It forms the base for model building.

Feature Scaling

Feature scaling standardizes feature values. Some features have larger ranges than others. This can affect model performance. StandardScaler is used to normalize data. It transforms data to mean 0 and standard deviation 1. Scaling is important for algorithms like SVR and KNN. It improves convergence speed. It ensures fair contribution of all features.

Machine Learning Algorithms Used

1. Linear Regression

Linear Regression predicts continuous values using a linear relationship. It fits a straight line to data. It minimizes prediction error. It is simple and easy to implement. It works well for linear data. It helps understand feature impact. However, it cannot capture complex patterns. It is sensitive to outliers. It is used as a baseline model.

2. Ridge Regression

Ridge Regression adds regularization to Linear Regression. It penalizes large coefficients. This reduces overfitting. It handles multicollinearity well. It keeps all features but reduces impact. It improves model stability. It balances bias and variance. It performs well on complex datasets.

3. Lasso Regression

Lasso Regression uses L1 regularization. It can reduce some coefficients to zero. This performs feature selection. It simplifies the model. It improves interpretability. It reduces overfitting. It is useful for high-dimensional data. It requires proper tuning.

4. Decision Tree Regressor

Decision Tree splits data into branches. It creates a tree-like structure. Each node represents a decision. It captures non-linear relationships. It is easy to understand. It does not require scaling. However, it may overfit. Pruning improves performance.

5. Random Forest Regressor

Random Forest uses multiple decision trees. It combines their outputs. It reduces overfitting. It improves accuracy. It handles large datasets well. It provides feature importance. It is reliable and widely used. It requires more computation.

6. Support Vector Regressor (SVR)

SVR fits data within a margin. It handles high-dimensional data. It uses kernel functions. It works well for complex patterns. It requires scaling. It is sensitive to parameters. It can be slow. It provides accurate results when tuned.

7. K-Nearest Neighbors (KNN) Regressor

KNN predicts based on nearest neighbors. It uses distance metrics. It is simple and intuitive. It works well with small datasets. It captures local patterns. It requires scaling. It is sensitive to K value. It becomes slow for large datasets.

8. Gradient Boosting Regressor

Gradient Boosting builds models sequentially. Each model corrects previous errors. It uses decision trees as base learners. It handles complex patterns well. It provides high accuracy. It uses gradient descent optimization. It requires parameter tuning. It can overfit if not controlled. It is widely used in real-world problems.

9. AdaBoost Regressor

AdaBoost focuses on difficult data points. It assigns higher weights to errors. Models are built sequentially. It improves weak learners. It is simple and efficient. It reduces bias and variance. It is sensitive to noise. It works best with clean data.

10. Extra Trees Regressor

Extra Trees adds randomness to tree building. It selects random splits. It reduces variance. It improves speed. It handles large datasets well. It prevents overfitting. It is efficient and stable. It may slightly reduce accuracy.

11. XGBoost Regressor

XGBoost is an advanced boosting algorithm. It is fast and efficient. It uses parallel processing. It includes regularization. It handles missing values. It provides very high accuracy. It prevents overfitting. It requires tuning. It is widely used in industry.

12. Polynomial Regression

Polynomial Regression models non-linear relationships. It adds higher-degree features. It fits curves instead of lines. It improves flexibility. It captures complex patterns. It may overfit with high degree. It requires careful selection. It is easy to implement.

Model Evaluation

Models are evaluated using R^2 Score and RMSE. Lower RMSE and higher R^2 indicate better performance. Models are trained on training data and tested on unseen data.

Results

Different algorithms produce different results. Random Forest and XGBoost usually perform best. Linear models work for simple data. Boosting methods give high accuracy. KNN and SVR need tuning. The Decision Tree may overfit. Ensemble methods perform better overall.

Conclusion

This project shows how Machine Learning predicts house prices. Multiple algorithms were applied and compared. Ensemble methods gave better accuracy. Preprocessing and scaling improved performance. Each algorithm has strengths and weaknesses. Choosing the right model is important. This project gives practical ML experience.

What I Learned

Data preprocessing techniques

Feature scaling importance

Working of ML algorithms

Model training and testing

Evaluation using R^2 and RMSE

Comparing model performance

Handling real datasets

Improving accuracy

Writing structured ML code